Running Art 專案檢討報告

底圖「API KEY REQUIRED」問題處理 + 全專案缺失分析與解決方案

對象：justin121710/running-art（線上版 https://justin121710.github.io/running-art/） · 2026-10-23

摘要

本報告逐行審閱了 `index.html`（約 3,300 行）、`engine.js`（620 行）與 `gpsart/*.py`（約 3,300 行），並在無頭 Chromium 中實際執行驗證。重點如下：

- 地圖顯示 API KEY REQUIRED 的原因：CARTO 自 2026 年 9 月起停止提供免金鑰的點陣圖磚，沒帶 key 的請求一律回 403 並印上浮水印。已修正並推送到分支：沒有 key 時自動改用 OpenStreetMap 圖磚；填入免費 CARTO key 即恢復原本的深／淺色樣式；key 失效時會自動退回 OSM，地圖不會再出現浮水印。
- 共整理 50 項待處理問題（另有 1 項已在本次修正）：高嚴重度 11 項、中 23 項、低 16 項，分屬演算法、路網資料、運算引擎與效能、操作介面、匯出、維護、授權與安全七類。
- 最關鍵的演算法發現：(1) 路網超過 10,000 個節點時，走廊尋路會被整段跳過，改成忽略圖形的純最短路，大範圍路線品質因此驟降；(2) 剪除死路與「找路口」兩個功能在雙向圖上根本沒有作用（已實測確認）；(3) 尋路是逐段貪婪的局部最短路，這是「橫跳」與「繞遠路」的根本來源，建議改用 HMM／Viterbi 式的地圖匹配。
- 影響使用者的明確 bug：反轉方向後高程圖沒有跟著反轉、「取消修改」後爬升資料消失、「清空」後仍可用「取消修改」救回舊路線、部分提示在英文介面仍顯示中文、瀏覽器封鎖 localStorage 時整個 App 無法啟動等。

類別	高	中	低	代表問題
A 演算法	3	7	3	大路網跳過走廊、貪婪局部尋路、吻合度只算單向
D 路網資料	2	4	2	剪死路失效、含快速道路、忽略 foot=no
E 引擎與效能	2	3	1	Pyodide 跑在主執行緒卡畫面、啟動失敗後無法重試
U 操作介面	2	5	4	反轉後高程圖錯位、localStorage 被封鎖時整頁掛掉
X 匯出	1	2	1	GPX 每點寫死海拔 0
M 維護	1	2	3	文件與實際 repo 不符、回歸測試不在 repo 裡
L 授權與安全	0	0	2	CDN 腳本沒有 SRI（版權標示已在本次修正）

一、地圖「API KEY REQUIRED」：原因與永久解法

1. 原因

網站原本使用 `https://{s}.basemaps.cartocdn.com/dark_all/...`（淺色是 `light_all`）。CARTO 從 2026-09-25 起要求點陣圖磚必須在網址帶上 `?key=`，沒帶就回 HTTP 403，回應內容是一張印滿「API KEY REQUIRED」的圖。瀏覽器的 `<img>` 遇到 403 照樣會把這張圖畫出來，所以地圖看起來不是空白，而是滿版浮水印。匯出圖片的底圖也是同一個來源，所以會一起壞掉。

2. 已完成的修正（分支 claude/awesome-fermi-r6m2qs）

項目	內容
圖磚來源抽象化	`index.html` 新增 `TILE_PROVIDERS`（carto／osm）。每個來源各自定義網址、是否有 @2x、深淺色濾鏡、版權文字。地圖與匯出圖片共用同一份設定。
沒有 key 時	預設改用 OpenStreetMap 官方圖磚（免金鑰）。深色模式用 `invert + hue-rotate` 濾鏡做出深色底圖。
有 key 時	只要在 `index.html` 第 716 行 `const CARTO_KEY=''` 填入金鑰，就會用回原本的 CARTO dark_all／light_all（網址自動加上 `?key=`）。
key 失效時自動降級	`probeTiles()` 會在啟動時 fetch 一顆圖磚檢查狀態碼。key 過期、額度用完或網域不符時，自動切換到 OSM，不會再出現浮水印。
匯出圖片	匯出時抓同一個來源的圖磚。OSM 沒有 @2x，`tilePlan()` 會改抓下一級縮放，清晰度跟原本一樣。不支援 `ctx.filter` 的舊 Safari 用 `difference` 合成模擬反轉。
版權標示	原本關掉了 Leaflet 的版權欄（`attributionControl:false`），但 OSM 與 CARTO 的使用條款都要求顯示。現在地圖右下角有一行 9px 小字（壓在工具列下方的縫裡，不擋操作），匯出圖片底邊也會印上一行版權文字。
驗證	在無頭 Chromium 中模擬四種情境：無 key 深色、無 key 淺色、有效 key、無效 key（403）。四種都選對了來源、濾鏡和版權文字；匯出的 canvas 沒有被跨域污染，圖磚也全部載入成功。限制：這個執行環境連不到真正的 CARTO 和 OSM 伺服器，所以伺服器回應是模擬的。部署後請實際打開網站確認一次。

3. 你需要做的一步（想恢復原本 CARTO 樣式的話）

> 1. 到 https://carto.com/basemaps/apikey 免費申請金鑰（不需要 CARTO 帳號）。
> 2. 網站限制（Website restrictions）填 `justin121710.github.io`；要在本機測試的話，再加 `localhost`。金鑰會出現在公開的原始碼裡，有網域限制別人就拿不去用。
> 3. 把金鑰填進 `index.html` 第 716 行：`const CARTO_KEY='你的金鑰';`，然後 commit、push。
> 4. 注意：網頁不可以加 `Referrer-Policy: no-referrer`，否則有網域限制的 key 會被拒絕（目前沒有加）。CARTO 的免費額度：非商業用途每月 500 萬次圖磚請求，商業用途每月 100 萬次。
>
> 不申請也可以：網站現在已經能正常顯示 OSM 底圖。

4. 怎樣才算「永久」？各方案比較

沒有任何免費的第三方服務能保證永遠不改規則。這次的事件本身就是例子：CARTO 免金鑰用了很多年，一公告就停了。因此「永久解決」要靠兩件事：(1) 不依賴單一來源，壞掉時能自動切換（本次已經做到）；(2) 長期把圖資掌握在自己手上。

方案	金鑰	優點	缺點／風險	建議
CARTO + 免費 key	需要（免費）	維持原本的樣式；深色底圖最好看	政策可能再變；有額度上限	主要來源（已支援）
OSM 官方圖磚	不需要	官方提供、最穩定；有 CORS	使用政策禁止大量使用；沒有深色版（用濾鏡模擬）；沒有 @2x	自動備援（已預設）

OpenFreeMap	不需要	完全免費、沒有額度、開源，可自架；有 dark／positron 樣式	只有向量圖磚，要改用 MapLibre GL；匯出流程要重寫	長期首選
Protomaps PMTiles 自架	不需要	單一檔案放在自己的空間（例如 Cloudflare R2 免費額度），完全自主	台灣範圍的檔案約數百 MB，超過 GitHub Pages 單檔 100MB 的限制；要 MapLibre	最徹底的永久解
Stadia／MapTiler	需要（免費方案）	樣式多；Stadia 用網域驗證，網址不必帶 key	要註冊帳號；有額度	可當第三來源

建議路線：短期用「CARTO key + OSM 自動備援」（已完成）；中期把底圖換成 MapLibre + OpenFreeMap 向量圖磚，匯出改成用 MapLibre 的 `preserveDrawingBuffer` 截圖，或用 MapLibre 的離屏渲染；長期如果使用量變大，再自架 PMTiles。

二、演算法（gpsart/routing.py）

高 A1　路網超過 10,000 個節點時，走廊尋路被整段跳過

位置：`route_in_corridor()` → `solve_sub()` 開頭的 `G_full.number_of_nodes() > 10000`。

問題：為了避免 O(N) 掃描太慢，大圖直接在整張圖上用 `weight='length'` 跑最短路。這樣既不限制在走廊裡，也不看 `travel_weight`（人行道懲罰、走過的路懲罰都失效）。文件記錄台北路網約每平方公里 4,670 個節點，框 2～3 km² 就可能超過門檻，使用者會覺得「框大一點，路線品質突然變很差」，卻找不到原因。

解法：載入路網時用 `shapely.STRtree`（或 numpy 網格分桶）建一次空間索引，走廊只查索引候選點，就可以拿掉 10,000 這個門檻（見 A3）。

高 A2　尋路是逐段貪婪的局部最短路，這是橫跳與繞路的根本來源

位置：`solve_route()` 第 [C] 段：手繪線每 40m 取一個樣點，每一小段各自在 150m 緩衝區內求「目前節點 → 下一個樣點最近節點」的最短路，再把結果串起來。

問題：每一步只看下一個樣點，看不到整體。「最近節點」在道路兩側之間反覆換邊，就會左右橫跳；某一步選錯，後面就得繞回來。後面那一長串清理（去尖刺、去迴圈、截彎、剪折返、修補跳點）大多是在補這一步的副作用，參數彼此牽制，所以專案才會出現多次回歸。

解法：改成全域最佳化的地圖匹配（map matching）。具體做法：每個樣點取 k 個候選節點或邊（例如 30m 內最近的 5 個），用 Viterbi 動態規劃在所有候選組合中求總成本最低的一條。
・發射成本（樣點到候選點的距離） = d^2/σ^2
・轉移成本 = $|\text{路網距離} - \text{樣點直線距離}|/\beta$ + 轉彎與人行道懲罰
參考 Newson & Krumm（2009）的 HMM 地圖匹配，Valhalla 的 Meili 與 OSRM 的 match 服務也都用這個方法。這樣多數的後製清理都可以拿掉，參數也會大幅減少。

高 A3　走廊的節點掃描是 O(N) 的 Python 迴圈，而且每 40m 就掃一次

問題：5km 的路線約有 125 個樣點，每個樣點都要對整張圖的所有節點做 bbox 檢查加 `poly.contains(Point)`；150m 找不到還要用 500m 再掃一次。在瀏覽器的 Pyodide 裡，這是生成時間的主要來源。

解法：載入路網時把節點座標存成 numpy 陣列並建 STRtree，走廊查詢改用 `tree.query(poly)`，複雜度降到 O(log N + k)。

中 A4　「直線筆畫」走另一條未受控的慣性路徑

位置：`solve_inertia_path()`，條件是筆畫直線度大於 0.95 且長度大於 50m。

問題：(1) 在整張圖上搜尋，不限制在走廊內，可能離畫的線很遠；(2) 每次 push 都複製整條路徑串列（`path + [v]`），剪枝條件 `min_costs[u] < cost*0.8` 又允許同一點被反覆展開，大圖上可能跑很久；(3) 產生的結果完全跳過後面的清理步驟，品質不一致。

解法：限制在走廊子圖內；改用以「（節點, 進入邊）」為狀態的標準 Dijkstra，把轉彎成本放進邊權重，用 parent 指標回溯路徑。如果 A2 改成 HMM，這條特殊路徑可以直接拿掉。

中 A5　截彎取直用「每節點 5 公尺」估路長，也不檢查是否偏離手繪線

位置：`advanced_shortcut_optimizer()`：`original_path_est = k*5`，判斷式是 `len(shortcut_path) < k`。

問題：節點數少不代表比較短，也不代表比較貼近原圖。捷徑用 `length` 計算，忽略人行道懲罰；而且沒有檢查捷徑離手繪線多遠，只靠 `span_cap`（最多跳過 1/4）防止整圈被抄掉。圖案的小細節（例如愛心的凹口）還是可能被抄平。

解法：用真實的累積長度比較；只有在捷徑每一點都落在手繪線 25～30m 以內時才採用。

中 A6　很多距離門檻用「度」計算，東西向和南北向的尺度不一樣

位置：走廊緩衝 `0.0015°`、尖刺門檻 `0.0004°`、`prune_final_spurs` 的 `*111000`、縫合與閉合判斷用的「度²」、RDP 平滑的 `tolerance/111000`。

問題：經度 1 度的實際長度是緯度 1 度乘上 cos(緯度)。在台灣（約北緯 25 度）差 9%，到高緯度地區差更多，門檻會跟著方向變。

解法：載入時就把座標投影到以範圍中心為基準的平面公尺座標（等距圓柱投影乘 $\cos(\text{lat}_0)$，或 UTM），後續所有幾何運算都在公尺座標上做。

中 A7　用「真假值」判斷節點是否存在，節點編號 0 會被當成「沒有節點」

位置：`if not src_n`、`if start_n and ...`、`if current_node and target_start_node`、`if end_node:`、`if not prev_end_node` 等，散布在多處。

問題：OSM 的節點編號是很大的正整數，目前不會出事。但 ARCHITECTURE.md 記載「路口合併」會把節點重新編號成 0..N，而合併啟用後出現了「原因不明的異常」。這時編號為 0 的節點會被靜靜當成 None，很可能就是原因之一，至少是線索。

解法：全部改成 `is None` 判斷；重新啟用路口合併前，先加上一個以編號 0 為起點的測試。

中 A8　吻合度只算單向，數值容易虛高

位置：`engine.js` 的 `_pack()`：只算「路線取樣點有幾成落在手繪點 40m 內」。

問題：(1) 只看「路線是否靠近圖」，沒看「圖有沒有被路線蓋到」，所以只跑了半個圖也可能顯示 100%；(2) 比的是到手繪「點」的距離而不是到「線段」的距離，滑鼠畫得快、點稀疏時會低估；(3) 雙層 Python 迴圈，長路線很慢。

解法：同時算準確率（路線落在圖附近的比例）與覆蓋率（圖被路線覆蓋的比例），兩者取 F1 或取較小值；距離改用 shapely 的線段距離或 buffer 判斷。也可以另外顯示 Fréchet／Hausdorff 距離（公尺）作為形狀誤差。

中 A9　多筆畫時沒有最佳化筆畫順序

問題：`solve_route` 依使用者下筆的順序處理筆畫，只會個別反轉或旋轉；寫好的 `optimize_stroke_order()` 完全沒被呼叫。畫文字或多個圖形時，連接段（灰色虛線）可能跨來跨去，總距離明顯變長。

解法：把它當成「起點 → 各筆畫（可正可反） → 終點」的 TSP 變形：先用貪婪法排出順序，再用 2-opt 改進。距離用路網最短路距離，或至少用直線距離近似。

中 A10　所有閉合圖形都被強制改成順時針

位置：`reorder_closed_strokes()` 的 [Phase 52] Force Clockwise。

問題：使用者無法讓某個圈逆時針跑，例如想讓兩個圈的方向一致、或配合坡度方向。「反轉」按鈕會把整條路線一起反轉，沒辦法只反轉其中一圈。

解法：預設保留使用者下筆的方向；在設定中提供「統一順時針」選項；微調模式加上「反轉這一段」。

低 A11　縫合門檻依圖形大小放大到最多 120m，可能誤縫本來就分開的筆畫

問題：寫字時，字母之間的空隙可能小於整體對角線的 4%，就會被縫成一筆，筆畫之間被硬拉一條線。

解法：只有在兩端方向接近時才縫（例如夾角小於 60 度），或提供「不要自動連接」的開關。

低 A12　約 1,000 行死程式碼，其中還有一個座標寫反的函式

內容：`solve_chinese_postman`、`_solve_cpp_core`、`patch_micro_gaps`、`get_line_hugging_path`、`optimize_stroke_order`、`check_continuity`、`remove_zigzags`（空殼）、`crop_graph_buffer`、`calculate_segments_distance` 都沒有被呼叫。`get_winding_angle` 把 (lat, lon) 當成 (x, y)，旋轉方向的正負號是相反的。另外有多處 `except Exception: pass`。

解法：移到 `archive/` 或直接刪除，降低閱讀和維護成本；保留的例外處理至少要記錄 log。

低 A13　「走過的路」懲罰會一直累積到整次生成結束

問題：同一次生成內，前一筆畫走過的路，後一筆畫再經過時也會被罰 4 倍。兩個圖形共用一條街（例如 8 字形的交叉處）時，後畫的那筆會被推去繞路。

解法：懲罰只套用在同一筆畫內；或者讓懲罰隨距離遞減。

三、路網資料與前處理（gpsart/graphprep.py、engine.js）

高 D1　「剪除死路」實際上完全沒有作用（已實測）

位置：`remove_recursive_dead_ends()` 用 `G.degree()==1` 找死路端點。

問題：建圖時每條路都加了雙向兩條邊（`MultiDiGraph`），所以死路端點的 degree 是 2（一進一出），不是 1。實測：鏈狀圖 1-2-3-4 加上 3-5，degree 分別是 {1:2, 2:4, 3:6, 4:2, 5:2}，沒有任何節點的 degree 是 1。文件說前處理會「剪除死路」，但實際上一個都沒剪。

解法：改用「不重複的鄰居數」判斷：`len(set(G.successors(n)) | set(G.predecessors(n))) == 1`。但修正前要先想清楚：全部剪掉會把死巷（很多人家門口就是死巷）一起刪掉，起訖點會被吸到很遠的地方。建議只剪長度小於 30～50m、而且不包含起訖點的短分岔。改完要跑基準測試。

高 D2　路網包含快速道路（trunk），也沒有看 foot=no

位置：`WALKABLE` 集合包含 `trunk`、`trunk_link`；只檢查 `access`。

問題：台灣的快速道路、部分市區高架在 OSM 常標成 `trunk`，行人禁止進入。另外：`foot=no`、`motorroad=yes` 沒有排除；`access=private + foot=yes`（允許行人）反而被擋掉。這直接關係到跑者安全（免責聲明也提到這點）。Overpass 查詢下載了所有 highway（包含 motorway），浪費頻寬。

解法：查詢改為 `way["highway"]["highway"!~"motorway|motorway_link|construction|proposed"]["foot"!~"no|private"]["motorroad"!="yes"]`。建圖時依 `foot` 覆寫 `access`。trunk 預設排除，或給很高的懲罰；primary／secondary 也加上適度懲罰。

中 D3　「附近路口」的篩選其實包含了所有非端點節點

位置：`engine.js` 的 `_nearby_junctions()`：`G.degree(n) >= 3`。

問題：原因和 D1 一樣：一般的路中間節點 degree 是 4，所以拖曳節點時出現的「候選路口」其實包含彎道上的形狀點，跟註解說的「只留路口」不符。

解法：改成不重複鄰居數 ≥ 3。

中 D4　簡化直線節點時，合併後的邊只保留第一條邊的屬性

位置：`remove_redundant_straight_nodes()` 只取 key 0，新邊直接複製 d1 的屬性。

問題：人行道接上行人穿越道合併後，`footway=crossing` 標籤可能消失，或者整段都變成 crossing，懲罰就算錯了；平行的多重邊也會遺失。

解法：只有在兩條邊的 `highway`／`footway` 都相同時才合併；或者合併時直接加總兩邊的 `travel_weight`，不要事後重算。

中 D5　同時向 6 台 Overpass 鏡像發請求，違反公用服務的使用規範

位置：`fetchOverpass()` 用 `Promise.any` 同時打 6 台。

問題：每次載入都對公用伺服器造成 6 倍負載，容易被限流（429）甚至封 IP，長期反而降低成功率。另外 UI 沒有限制框選面積，框很大時會逾時，也會觸發 A1。

解法：改成錯開競速：先打 1 台，3 秒沒回應再加 1 台，依此類推；遇到 429 要遵守 `Retry-After`。查詢加上 `[maxsize:]`。框選時即時顯示 km²，並設上限（例如 6 km²），超過就提示分區載入。

中 D6　本地快取永不過期，也不會挑最小的範圍

位置：`cacheFind()`／`cachePut()`。

問題：OSM 資料會更新（封路、新路），但快取永遠沿用舊資料。只要找到任何一個「涵蓋」新範圍的快取就用，可能用到一塊很大的範圍，建圖變慢、也更容易超過 10,000 節點。

解法：加上 TTL（例如 30 天）；有多個符合時選面積最小的；超過 TTL 但離線時才沿用舊資料。

低 D7　只保留最大連通塊，其他區域被靜靜丟掉

問題：如果河濱公園和市區之間的橋沒有畫進 OSM，使用者在公園裡畫的圖會被吸到很遠的地方，而且沒有任何提示。

解法：計算被丟掉的區域，在地圖上用灰色標示「這裡的道路沒有連到主路網」。

低 D8　台灣的人行道常獨立成另一條線，路網加倍

問題：人行道約占全部邊的 44%（文件中的數據）。目前靠 `FOOTWAY_PENALTY` 壓制，但這同時讓沒有車道的步道也被懲罰。

解法：只懲罰與車道平行（距離 15m 以內、方向一致）的 footway，獨立步道不罰。

四、運算引擎與效能（engine.js）

高 E1　Pyodide 跑在主執行緒，生成時整個畫面凍結

問題：`runPythonAsync` 雖然叫 async，但 Python 是同步執行，生成的 1.8～3.8 秒（長路線更久）畫面完全不能互動。手機上可能跳出「網頁沒有回應」，也沒辦法取消。

解法：把 Pyodide 放進 Web Worker（官方支援），用 postMessage 或 Comlink 呼叫；進度回報改成由 Worker 傳訊息；提供「取消」按鈕（直接 terminate Worker）。

高 E2　引擎載入失敗一次，之後就永遠無法重試

位置：`boot()` 把 `readyPromise` 存起來，失敗的 promise 也一樣存著。

問題：第一次因為網路不穩載入失敗後，之後所有 `loadArea`、`generate` 都直接失敗，只能重新整理頁面，而使用者不會知道要這樣做。

解法：catch 時把 `readyPromise` 設回 null，並在錯誤提示上加「重試」按鈕。

中 E3　首次載入太重，而且從 PyPI 安裝套件

問題：networkx、shapely 用 `micropip.install` 從 PyPI 安裝，但 Pyodide 發行版本身就內建這兩個套件，可以直接 `loadPackage` 從同一個 CDN 載入，比較快也比較穩。沒有 Service Worker，離線時無法啟動引擎（地圖快取雖然支援離線，但引擎本身做不到）。

解法：改成 `py.loadPackage(['numpy','networkx','shapely'])`；加上 Service Worker，快取 Pyodide、套件和 gpsart 檔案；進站後先背景預載。

中 E4　沒有防止重複執行

問題：生成中可以再按一次「生成」、反轉或拖曳節點。多個 Python 呼叫共用 `strokes_json`、`SEGS` 等全域變數，可能互相覆蓋。

解法：引擎內部用一個佇列（mutex）讓呼叫依序執行；忙碌時 UI 停用相關按鈕。

中 E5　Python 膠水程式寫在 JS 的字串裡

問題：`PY_SETUP` 約 290 行 Python 放在 template string 裡，不能 lint、不能單元測試，基準測試也守不到（文件說這是刻意的，但微調功能的 bug 就沒有測試保護）。

解法：移到 `gpsart/web_api.py`，和其他模組一起 fetch，並加上測試。

低 E6　拖曳節點時，每次都用 Python 迴圈掃描所有節點找附近路口

解法：和 A3 共用空間索引。

五、操作與介面（index.html）

高 U1　瀏覽器封鎖 localStorage 時，整個 App 無法啟動

位置：頂層的 `localStorage.getItem('runart_arrow_r')`、`'runart_arrow'`、`'runart_theme'`、`'runart_lang'`、`'runart_pace'` 等都沒有 try。

問題：Chrome 的「封鎖所有 Cookie」設定、某些內嵌 WebView、iframe 嵌入時，存取 localStorage 會直接丟 SecurityError，中斷整段主程式，畫面只剩一張地圖、所有按鈕都沒有反應。免責聲明那一段有 try，主程式沒有，前後不一致。

解法：寫一個 `store.get/set` 包裝函式，內部用 try/catch，所有存取都改用它。

高 U2　反轉方向後，高程圖沒有跟著反轉

位置：`$('aRev').onclick`：只對調了爬升和下降的數字，沒有反轉 `elevChartData.elevations`。

問題：反轉後地圖上的跑者從另一頭出發，但高程圖還是舊方向，拖曳時跑者在地圖上的位置和高程圖上的點對不起來；匯出圖片的高程區也是反的。

解法：`elevChartData.elevations.reverse(); renderElevChart(elevChartData);`

中 U3　「取消修改」後，爬升與高程圖消失

位置：`btnCancelMod`：`dropRoute()` 已經把 `elevGain` 清掉，還原路線時沒有重新計算。

解法：還原後呼叫 `updateElevation()`；或者在進入修改模式時把高程資料一起存在 `prevResult`。

中 U4　按「清空」後，仍可以用「取消修改」救回舊路線

位置：`clearAll()` 沒有清掉 `S.prevResult`（只有「重新繪製」會清）。

問題：修改模式中按「清空」，筆畫和起訖點都清掉了，但「取消修改」按鈕還在，按下去會跑出一條沒有手繪線、也沒有起訖點的舊路線，狀態不一致。

解法：在 `clearAll()` 裡設 `S.prevResult=null`。

中 U5　英文介面下仍有中文提示

問題：`TOASTS_EN` 的對照表是用中文原句當 key，但呼叫端的文字已經改過，對不上：刪除節點失敗的兩句（「…已自動取消選取…」）、進入微調模式的提示（「點一下節點可選取；直接拖曳…」）都會顯示中文。

解法：全部改用 `tx(zh,en)` 寫法，不再靠字串比對；或者改成 key 式的 i18n 表。

中 U6　匯出預覽在某些勾選組合下不會查詢地點名稱

位置：`renderPreview()` 只有在 `showStats` 時才查地點。

問題：關掉所有數據、只留「地點」時，預覽沒有地名，但最後匯出的圖有，預覽和成品不一致。

解法：條件改成 `showPlace`。

中 U7　框選範圍沒有面積上限，也沒有即時面積顯示

問題：框太大時會下載很久、逾時，或者超過 10,000 節點（A1），使用者不知道為什麼。

解法：拖曳時在框旁邊即時顯示 km²，以及預估下載時間（`export.py` 已經有估算公式）；超過上限時框變成紅色，並提示縮小範圍。

低 U8　禁止縮放頁面（user-scalable=no）

問題：不符合無障礙準則 WCAG 1.4.4，視力不好的使用者無法放大字體。地圖本身由 Leaflet 處理手勢，不受影響。

解法：移除 `maximum-scale=1, user-scalable=no`，改用 CSS `touch-action` 控制地圖區域。

低 U9　很短的筆畫會被靜靜丟掉；起訖點連續點擊有競態

問題：點數不超過 3 的筆畫直接移除，沒有任何提示。起訖點吸附是非同步的，快速點兩次時，先點的那次可能比較晚完成，蓋掉後點的。

解法：短筆畫丟掉時提示「筆畫太短」；吸附加上序號，只採用最後一次的結果。

低 U10　沒有地點搜尋和定位，起始位置固定在台北

解法：加上搜尋框（Nominatim search）與「定位我」按鈕（`map.locate()`）。

低 U11　重新整理頁面後作品全部消失，也不能分享

解法：把筆畫、起訖點、路線存進 IndexedDB，進站時詢問是否還原；提供分享連結（把筆畫壓縮成 polyline 字串放進網址 hash）。

六、匯出（圖片與 GPX）

高 X1　GPX 每個點都寫死 <ele>0.0</ele>

位置：`engine.js` 的 `toGPX()`，以及 `export.py`。

問題：Strava、Garmin Connect、Komoot 等平台匯入時，有些會把它當成真實海拔，顯示爬升 0m 或平坦的剖面；導航錶也可能拿它計算爬升預估。

解法：直接省略 `<ele>`（讓平台用自己的 DEM 計算）；或者用已經查到的 Open-Meteo 高程，沿線內插後填入。

中 X2　GPX 有重複點，也缺少起訖標記

問題：相鄰分段的接點會重複寫入，部分裝置會因此算出 0 距離的點，或產生多餘的轉向提示。

解法：寫入前去除連續重複點；加上起點、終點 `<wpt>`；提供 Garmin Course（TCX）或 `<rte>` 格式選項。

中 X3　匯出和預覽各寫一份繪圖程式

問題：`renderExportCanvas` 和 `renderPreview` 各有一份 TH 主題物件、路線、已跑段、箭頭的繪製程式，大約重複 150 行。註解中已經記錄過好幾次兩邊不一致的事件，以後還會再發生。

解法：合併成一個 `renderScene(ctx, size, opt, {fetchTiles})`，預覽和匯出只差在尺寸和是否抓圖磚。本次修正已經把圖磚規劃抽成共用的 `tilePlan()`，可以沿著這個方向繼續。

低 X4　爬升直接用 100 個取樣點加總，雜訊會灌水

解法：先平滑，或加上 2～3m 的遲滯門檻（落差累積超過門檻才算爬升），數值會更接近運動手錶的結果。

七、維護與文件

高 M1　回歸測試不在這個 repo 裡，也沒有 CI

問題：文件反覆強調「動 gpsart/ 就要跑 `selftest_algorithms.py compare`」，但這個 repo 裡沒有 `selftest_algorithms.py`、`fixture_graph.graphml`、`baseline.json`。部署用的程式碼完全沒有自動化保護，而這個專案已經發生過兩次嚴重回歸。

解法：把測試、fixture、baseline 加進 repo；建立 GitHub Actions：push 時用 CPython 跑基準測試，並用 Playwright 跑一次「框選、畫圖、生成、匯出」的端對端煙霧測試（Overpass 回應可以用錄好的 fixture 模擬）。

中 M2　文件描述的資料夾結構和實際 repo 不一致

問題：README.md 和 ARCHITECTURE.md 提到 `deploy/`、`web_prototype/`、「gpsart 有三份副本要一起同步」、`cd deploy && git push`，但這個 repo 的根目錄本身就是部署內容。新加入的人會被誤導。

解法：以這個 repo 的實際結構重寫「資料夾」和「部署」兩章。

中 M3　單一 3,300 行的 HTML，大量共用全域狀態

解法：不需要打包工具，直接用原生 ES modules 拆成 `map.js`、`draw.js`、`edit.js`、`export.js`、`i18n.js`、`theme.js`，狀態集中在一個 store。

低 M4　快取版本號要手動改

問題：`VERSION` 和 `engine.js?v=` 都要人工更新，忘記改就會讓使用者跑舊程式。

解法：由 CI 依檔案內容雜湊自動替換；或者交給 Service Worker 管理版本。

低 M5　大量 except Exception: pass

解法：至少記錄 `debug_log`；可以預期的錯誤改成捕捉特定例外（例如 `nx.NetworkXNoPath`）。

低 M6　config.py 留有桌面版用的設定

內容：`TAIWAN_LOCATIONS`、`OUTPUT_ROOT`、下載時間估算等，網頁版沒有用到；`route_in_corridor` 的 docstring 寫 40m／100m，實際是 150m／500m。

解法：清掉沒用的設定，修正 docstring。

八、授權與安全

已修正 L1　地圖與匯出圖片沒有顯示圖資版權

OSM 的 ODbL 授權與 CARTO 的服務條款都要求顯示「© OpenStreetMap contributors」（以及 © CARTO）。本次已經在地圖右下角和匯出圖片底邊加上。

低 L2　第三方 CDN 腳本沒有 SRI

問題：Leaflet（unpkg）和 Pyodide（jsDelivr）沒有 `integrity` 屬性，CDN 一旦被竄改，就能在使用者的瀏覽器執行任意程式。

解法：加上 `integrity="sha384-…" crossorigin="anonymous"`；或者把 Leaflet 放進 repo 自己提供。

低 L3　公用服務的使用政策

Nominatim 規定每秒最多 1 次請求、禁止大量查詢（目前有快取，可以接受）；OSM 圖磚禁止大量使用和預先抓取。如果之後使用量變大，請改用自架服務或付費方案（見第一章的方案比較）。CARTO 的 key 一定要設網域限制。

九、建議執行順序

階段	項目	預期效果
已完成	底圖來源抽象化、OSM 自動備援、CARTO key 支援、版權標示（L1）	地圖不再出現 API KEY REQUIRED
短期（1 週）小改動	U1 localStorage 防呆、U2 反轉高程、U3、U4、U5、U6、E2 引擎重試、D3 路口篩選、A7 改用 `is None`、X1 移除 ele=0、X2 去重複點、L2 SRI	消除明確 bug，每項大約 1～20 行
中期（1 個月）	M1 測試與 CI、E1 Web Worker、A1＋A3 空間索引、D2 道路過濾、D5 Overpass 禮儀＋U7 面積上限、D6 快取 TTL、E3 Service Worker、X3 合併繪圖程式、M2 更新文件	大範圍品質不再驟降、畫面不卡、更安全的路線
長期	A2 HMM／Viterbi 地圖匹配（取代大部分後製清理）、A8 雙向吻合度、A9 筆畫順序最佳化、A10 方向選項、D1 短分岔修剪、底圖改用 MapLibre＋OpenFreeMap／自架 PMTiles、U10、U11	路線品質根本性提升、底圖完全自主

> 改演算法前的提醒：D1、D2、A1、A2、A5、A6 都會改變路線結果。請先把基準測試加回 repo（M1），用固定的路網 fixture 比較前後的覆蓋率、橫跳次數、總長度和吻合度，確認是改善才合併。